

EXAMENES PYTHON

```
import pytest
```

```
def test_function():  
    assert response == 200  
    grade = score / total  
    == 10 + 20
```



Índice general

1. Exámenes basicos (1-10)	5
1.1. Examen 1: Variables y operaciones	5
1.2. Examen 2: Condicionales	6
1.3. Examen 3: Bucle while	9
1.4. Examen 4: Bucle for y range	11
1.5. Examen 5: Listas basicas	12
1.6. Examen 6: Metodos de listas	15
1.7. Examen 7: Diccionarios	16
1.8. Examen 8: Funciones	18
1.9. Examen 9: Modulo random	20
1.10. Examen 10: Proyecto Cajero Automatico	23
2. Exámenes intermedios (11-20)	25
2.1. Examen 11: Metodos de strings	25
2.2. Examen 12: Matrices	26
2.3. Examen 13: Comprension de listas	29
2.4. Examen 14: Carrito de compras	30
2.5. Examen 15: Modulo os basico	33
2.6. Examen 16: Ahorcado	34
2.7. Examen 17: Generador de contraseñas	37
2.8. Examen 18: Sistema de notas	39
2.9. Examen 19: Try/Except	41
2.10. Examen 20: Agenda de contactos	43
3. Exámenes avanzados (21-30)	45
3.1. Examen 21: Bucles anidados	45
3.2. Examen 22: Piedra Papel Tijera	46
3.3. Examen 23: Biblioteca	49
3.4. Examen 24: Recursividad	51
3.5. Examen 25: Lista de tareas	53
3.6. Examen 26: Conversor de unidades	55
3.7. Examen 27: Deteccion de errores	57
3.8. Examen 28: Variables globales y locales	59
3.9. Examen 29: Modulos	60
3.10. Examen 30: Supermercado con descuentos	62

Capítulo 1

Exámenes básicos (1-10)

1.1. Examen 1: Variables y operaciones

Preguntas teóricas (5)

1. ¿Qué función se usa en Python para mostrar información en pantalla?
2. ¿Qué función se usa para pedir datos al usuario?
3. ¿Qué tipo de dato devuelve la función `input()` siempre?
4. ¿Cómo se convierte un string a un número entero?
5. ¿Cuál es la diferencia entre `/` y `//`?

Ejercicios prácticos (5)

Ejercicio

1. Escribe un programa que pida el nombre del usuario y luego muestre “Hola [nombre]”.

Ejercicio

2. Pide dos números enteros y muestra su suma, resta, multiplicación y división.

Ejercicio

3. Pide un número decimal y muestra su doble y su triple.

Ejercicio

4. Pide el año de nacimiento y calcula la edad en 2026.

Ejercicio

5. Pide un número de horas y muestra cuántos minutos y segundos son.

Soluciones

Solucion

Teoricas:

1. `print()`
2. `input()`
3. `string (texto)`
4. `int()`
5. `/` da division decimal, `//` da division entera (trunca)

Practicas:

```
1 # 1
2 nombre = input("Nombre: ")
3 print("Hola", nombre)
4
5 # 2
6 a = int(input("A: "))
7 b = int(input("B: "))
8 print("Suma:", a+b)
9 print("Resta:", a-b)
10 print("Multiplicacion:", a*b)
11 print("Division:", a/b)
12
13 # 3
14 num = float(input("Numero: "))
15 print("Doble:", num*2)
16 print("Triple:", num*3)
17
18 # 4
19 anio = int(input("Anio nacimiento: "))
20 edad = 2026 - anio
21 print("Edad:", edad)
22
23 # 5
24 horas = int(input("Horas: "))
25 minutos = horas * 60
26 segundos = horas * 3600
27 print(horas, "horas son", minutos, "minutos y", segundos, "segundos")
```

1.2. Examen 2: Condicionales

Preguntas teoricas (5)

1. ¿Que operador se usa para comparar si dos valores son iguales?

2. ¿Que significa el operador %?
3. ¿Cual es la diferencia entre = y ==?
4. ¿Que hacen `and` y `or`?
5. ¿Cuántas veces se puede usar `elif` en una estructura?

Ejercicios practicos (5)

Ejercicio

1. Pide un numero y muestra si es positivo, negativo o cero.

Ejercicio

2. Pide un numero y muestra si es par o impar.

Ejercicio

3. Pide la edad y muestra si es mayor de edad (18 o mas).

Ejercicio

4. Pide dos numeros y muestra el mayor.

Ejercicio

5. Pide una letra y muestra si es vocal o consonante.

Soluciones

Solucion

Teoricas:

1. ==
2. Modulo, devuelve el resto de la division entera.
3. = es asignacion, == es comparacion.
4. and ambas condiciones deben ser True; or al menos una.
5. Las veces que quieras.

Practicas:

```
1 # 1
2 n = int(input("Numero: "))
3 if n > 0:
4     print("Positivo")
5 elif n < 0:
6     print("Negativo")
7 else:
8     print("Cero")
9
10 # 2
11 n = int(input("Numero: "))
12 if n % 2 == 0:
13     print("Par")
14 else:
15     print("Impar")
16
17 # 3
18 edad = int(input("Edad: "))
19 if edad >= 18:
20     print("Mayor de edad")
21 else:
22     print("Menor de edad")
23
24 # 4
25 a = int(input("A: "))
26 b = int(input("B: "))
27 if a > b:
28     print("Mayor:", a)
29 elif b > a:
30     print("Mayor:", b)
31 else:
32     print("Son iguales")
33
34 # 5
35 letra = input("Letra: ").lower()
36 if letra in "aeiou":
37     print("Vocal")
38 else:
39     print("Consonante")
```

1.3. Examen 3: Bucle while

Preguntas teoricas (5)

1. ¿Que es un bucle infinito y como se evita?
2. ¿Que instruccion se usa para salir de un bucle antes de tiempo?
3. ¿Que hace `continue` dentro de un bucle?
4. ¿Cuales son las tres partes fundamentales de un bucle while con contador?
5. ¿Que pasa si no se actualiza la variable de control dentro del while?

Ejercicios practicos (5)

Ejercicio

1. Muestra los numeros del 1 al 10 usando while.

Ejercicio

2. Muestra los numeros pares del 2 al 20.

Ejercicio

3. Suma los numeros del 1 al 100 y muestra el resultado.

Ejercicio

4. Pide numeros al usuario hasta que ingrese 0, luego muestra la suma total.

Ejercicio

5. Pide una contrasena hasta que sea "python123".

Soluciones

Solucion

Teoricas:

1. Bucle que nunca termina; se evita actualizando la variable de control.
2. `break`
3. Salta a la siguiente iteracion sin ejecutar el resto del bloque.
4. Inicializacion, condicion, actualizacion.
5. Se produce un bucle infinito.

Practicass:

```
1  # 1
2  i = 1
3  while i <= 10:
4      print(i)
5      i += 1
6
7  # 2
8  i = 2
9  while i <= 20:
10     print(i)
11     i += 2
12
13 # 3
14 suma = 0
15 i = 1
16 while i <= 100:
17     suma += i
18     i += 1
19 print("Suma:", suma)
20
21 # 4
22 suma = 0
23 n = int(input("Numero: "))
24 while n != 0:
25     suma += n
26     n = int(input("Numero: "))
27 print("Suma total:", suma)
28
29 # 5
30 clave = input("Contrasena: ")
31 while clave != "python123":
32     print("Incorrecta")
33     clave = input("Contrasena: ")
34 print("Acceso concedido")
```

1.4. Examen 4: Bucle for y range

Preguntas teoricas (5)

1. ¿Que hace la funcion `range(5)`?
2. ¿Cual es la diferencia entre `range(1,6)` y `range(1,6,2)`?
3. ¿Como se recorre un string letra por letra con for?
4. ¿Que significa que el for sea “autonomo”?
5. ¿Se puede usar `break` dentro de un for?

Ejercicios practicos (5)

Ejercicio

1. Muestra los numeros del 1 al 10 con for.

Ejercicio

2. Muestra los numeros pares del 2 al 20.

Ejercicio

3. Pide una palabra y muestra cada letra en una linea aparte.

Ejercicio

4. Suma los numeros del 1 al 100 usando for.

Ejercicio

5. Pide 5 frutas y las muestra (como en tu clase6).

Soluciones

Solucion

Teoricas:

1. Genera los numeros 0,1,2,3,4.
2. `range(1,6)`: 1,2,3,4,5; `range(1,6,2)`: 1,3,5.
3. `for letra in palabra: print(letra)`
4. No necesita una variable contador manual; el `for` la maneja internamente.
5. Si, `break` funciona en cualquier bucle.

Practicas:

```
1  # 1
2  for i in range(1, 11):
3      print(i)
4
5  # 2
6  for i in range(2, 21, 2):
7      print(i)
8
9  # 3
10 palabra = input("Palabra: ")
11 for letra in palabra:
12     print(letra)
13
14 # 4
15 suma = 0
16 for i in range(1, 101):
17     suma += i
18 print("Suma:", suma)
19
20 # 5
21 for i in range(5):
22     fruta = input("Fruta: ")
23     print("La fruta es:", fruta)
```

1.5. Examen 5: Listas basicas

Preguntas teoricas (5)

1. ¿Como se crea una lista vacia en Python?
2. ¿Que indice tiene el primer elemento de una lista?
3. ¿Que metodo agrega un elemento al final de una lista?
4. ¿Como se accede al ultimo elemento de una lista sin saber su longitud?

5. ¿Que hace `len(lista)`?

Ejercicios practicos (5)

Ejercicio

1. Crea una lista con los nombres de 3 amigos y muéstralos con un `for`.

Ejercicio

2. Crea una lista vacía, pide 3 nombres y agrégalos con `.append()`.

Ejercicio

3. Dada una lista de números, muestra el primero, el último y la cantidad.

Ejercicio

4. Modifica el segundo elemento de una lista (cambia “Pera” por “Sandía”).

Ejercicio

5. Crea dos listas vacías: una para pares y otra para impares. Pide 5 números y clasifícalos.

Soluciones

Solucion

Teoricas:

1. lista = []
2. 0
3. .append()
4. Usando indice -1: lista[-1]
5. Devuelve la cantidad de elementos de la lista.

Practicas:

```
1 # 1
2 amigos = ["Ana", "Luis", "Carlos"]
3 for a in amigos:
4     print(a)
5
6 # 2
7 nombres = []
8 for i in range(3):
9     nom = input("Nombre: ")
10    nombres.append(nom)
11 print(nombres)
12
13 # 3
14 nums = [10, 20, 30, 40, 50]
15 print("Primero:", nums[0])
16 print("Ultimo:", nums[-1])
17 print("Cantidad:", len(nums))
18
19 # 4
20 frutas = ["Manzana", "Pera", "Naranja"]
21 frutas[1] = "Sandia"
22 print(frutas)
23
24 # 5
25 pares = []
26 impares = []
27 for i in range(5):
28     n = int(input("Numero: "))
29     if n % 2 == 0:
30         pares.append(n)
31     else:
32         impares.append(n)
33 print("Pares:", pares)
34 print("Impares:", impares)
```

1.6. Examen 6: Metodos de listas

Preguntas teoricas (5)

1. ¿Que hace `list.insert(pos, elem)`?
2. ¿Cual es la diferencia entre `remove()` y `pop()`?
3. ¿Como se ordena una lista de menor a mayor?
4. ¿Como se invierte el orden de una lista?
5. ¿Que devuelve `lista.index(valor)`?

Ejercicios practicos (5)

Ejercicio

1. Usa `insert()` para agregar “Manzana” en la posicion 1 de una lista.

Ejercicio

2. Usa `pop()` para eliminar el ultimo elemento de una lista.

Ejercicio

3. Usa `remove()` para eliminar “Pedro” de una lista de nombres.

Ejercicio

4. Ordena una lista de numeros y luego invertela.

Ejercicio

5. Usa `count()` para contar cuantas veces aparece un numero en una lista.

Soluciones

Solucion

Teoricas:

1. Inserta un elemento en la posicion indicada (desplaza los siguientes).
2. `remove()` elimina por valor, `pop()` elimina por indice y devuelve el elemento.
3. `lista.sort()`
4. `lista.reverse()`
5. La posicion (indice) de la primera aparicion del valor.

Practicas:

```
1 # 1
2 frutas = ["Pera", "Banana"]
3 frutas.insert(1, "Manzana")
4 print(frutas)
5
6 # 2
7 nums = [1,2,3,4,5]
8 nums.pop()
9 print(nums)
10
11 # 3
12 nombres = ["Ana", "Pedro", "Luis"]
13 nombres.remove("Pedro")
14 print(nombres)
15
16 # 4
17 nums = [5,2,8,1,9]
18 nums.sort()
19 print(nums)
20 nums.reverse()
21 print(nums)
22
23 # 5
24 lista = [1,2,1,3,1,4]
25 print("Unos:", lista.count(1))
```

1.7. Examen 7: Diccionarios

Preguntas teoricas (5)

1. ¿Que es un diccionario en Python?
2. ¿Como se accede al valor asociado a una clave?
3. ¿Como se agrega un nuevo par clave:valor?

4. ¿Que diferencia hay entre `dict[key]` y `dict.get(key)`?
5. ¿Como se recorre un diccionario para obtener clave y valor?

Ejercicios practicos (5)

Ejercicio

1. Crea un diccionario con los datos de una persona (nombre, edad, ciudad).

Ejercicio

2. Agrega una nueva clave “profesion” al diccionario anterior.

Ejercicio

3. Modifica la edad a 26 y muestra solo la edad.

Ejercicio

4. Recorre el diccionario y muestra todas las claves y valores.

Ejercicio

5. Crea una agenda (diccionario) donde la clave sea el nombre y el valor el telefono. Agrega 3 contactos.

Soluciones

Solucion

Teoricas:

1. Coleccion de pares clave:valor, no ordenada.
2. `diccionario[clave]`
3. `diccionario[nueva_clave] = valor`
4. `dict.get(key)` devuelve `None` si la clave no existe, no lanza error.
5. `for clave, valor in dic.items():`

Practicas:

```
1 # 1
2 persona = {"nombre": "Ana", "edad": 30, "ciudad": "Madrid"}
3
4 # 2
5 persona["profesion"] = "Ingeniera"
6
7 # 3
8 persona["edad"] = 26
9 print(persona["edad"])
10
11 # 4
12 for k, v in persona.items():
13     print(k, ":", v)
14
15 # 5
16 agenda = {}
17 agenda["Juan"] = "123456"
18 agenda["Maria"] = "789012"
19 agenda["Pedro"] = "345678"
20 print(agenda)
```

1.8. Examen 8: Funciones

Preguntas teoricas (5)

1. ¿Que palabra clave se usa para definir una funcion?
2. ¿Que es una funcion pura?
3. ¿Cual es la diferencia entre `return` y `print` dentro de una funcion?
4. ¿Que es un parametro?
5. ¿Que significa que una funcion devuelva `None`?

Ejercicios practicos (5)

Ejercicio

1. Crea una funcion `suma(a,b)` que devuelva la suma.

Ejercicio

2. Crea una funcion `es_par(n)` que devuelva `True` si es par, `False` si no.

Ejercicio

3. Crea una funcion `maximo(a,b)` que devuelva el mayor de dos numeros.

Ejercicio

4. Crea una funcion `area_circulo(r)` que devuelva el area ($\pi * r^2$).

Ejercicio

5. Crea una funcion `invertir_cadena(cadena)` que devuelva la cadena al revés.

Soluciones

Solucion

Teoricas:

1. `def`
2. No tiene efectos secundarios (no imprime, no modifica globales), solo devuelve un valor.
3. `return` devuelve un valor; `print` solo muestra en pantalla.
4. Variable que recibe la funcion.
5. La funcion no tiene un `return` explicito o devuelve `None`.

Practicas:

```
1 # 1
2 def suma(a,b):
3     return a + b
4
5 # 2
6 def es_par(n):
7     return n % 2 == 0
8
9 # 3
10 def maximo(a,b):
11     if a > b:
12         return a
13     else:
14         return b
15
16 # 4
17 import math
18 def area_circulo(r):
19     return math.pi * r ** 2
20
21 # 5
22 def invertir_cadena(cad):
23     return cad[::-1]
```

1.9. Examen 9: Modulo random

Preguntas teoricas (5)

1. ¿Como se importa el modulo random?
2. ¿Que hace `random.randint(a,b)`?
3. ¿Que hace `random.choice(lista)`?

4. ¿Que hace `random.shuffle(lista)`?

5. ¿Que tipo de numeros genera `random.random()`?

Ejercicios practicos (5)

Ejercicio

1. Genera un numero aleatorio entre 1 y 10 y muestralo.

Ejercicio

2. Crea una lista de 5 colores y elige uno al azar.

Ejercicio

3. Simula 10 lanzamientos de un dado (1-6) y muestra los resultados.

Ejercicio

4. Juego de adivinanza: la PC elige un numero entre 1 y 20; el usuario tiene 5 intentos.

Ejercicio

5. Mezcla una lista de numeros y luego muestrala.

Soluciones

Solucion

Teoricas:

1. `import random`
2. Entero aleatorio entre a y b (inclusive).
3. Elige un elemento aleatorio de la lista.
4. Mezcla la lista (modifica la original).
5. Decimal entre 0 y 1 (sin incluir 1).

Practicas:

```
1 import random
2
3 # 1
4 print(random.randint(1,10))
5
6 # 2
7 colores = ["rojo","verde","azul","amarillo","negro"]
8 print(random.choice(colores))
9
10 # 3
11 for i in range(10):
12     print(random.randint(1,6))
13
14 # 4
15 secreto = random.randint(1,20)
16 for i in range(5):
17     intento = int(input("Adivina: "))
18     if intento == secreto:
19         print("Ganaste")
20         break
21     elif intento < secreto:
22         print("Mas alto")
23     else:
24         print("Mas bajo")
25 else:
26     print("Perdiste. Era", secreto)
27
28 # 5
29 nums = [1,2,3,4,5]
30 random.shuffle(nums)
31 print(nums)
```

1.10. Examen 10: Proyecto Cajero Automatico

Preguntas teoricas (5)

1. ¿Que variable global se usa para guardar el saldo?
2. ¿Por que se necesita `global saldo` dentro de las funciones de deposito y retiro?
3. ¿Que verificamos antes de restar dinero en la opcion retirar?
4. ¿Como mantenemos el menu repitiendose?
5. ¿Como se termina el programa?

Ejercicios practicos (5)

Ejercicio

1. Escribe la funcion `ver_saldo()`.

Ejercicio

2. Escribe la funcion `depositar()`.

Ejercicio

3. Escribe la funcion `retirar()`.

Ejercicio

4. Escribe el menu principal con `while`.

Ejercicio

5. Codigo completo del cajero (saldo inicial 5000).

Soluciones

Solucion

Teoricas:

1. saldo
2. Para modificar la variable global dentro de la funcion.
3. Que el monto no sea mayor al saldo y que sea positivo.
4. Con un bucle `while True` y `break` para salir.
5. Cuando el usuario elige la opcion Salir, se ejecuta `break`.

Practicas:

```
1 saldo = 5000
2
3 def ver_saldo():
4     print(f"Saldo: ${saldo}")
5
6 def depositar():
7     global saldo
8     monto = float(input("Monto: $"))
9     if monto > 0:
10         saldo += monto
11         print("Deposito exitoso.")
12     else:
13         print("Monto invalido.")
14
15 def retirar():
16     global saldo
17     monto = float(input("Monto: $"))
18     if monto > 0 and monto <= saldo:
19         saldo -= monto
20         print("Retiro exitoso.")
21     else:
22         print("Saldo insuficiente o monto invalido.")
23
24 while True:
25     print("\n--- CAJERO ---")
26     print("1. Ver saldo")
27     print("2. Depositar")
28     print("3. Retirar")
29     print("4. Salir")
30     op = input("Opcion: ")
31     if op == "1":
32         ver_saldo()
33     elif op == "2":
34         depositar()
35     elif op == "3":
36         retirar()
37     elif op == "4":
38         print("Gracias, vuelva pronto.")
39         break
40     else:
41         print("Opcion invalida.")
```

Capítulo 2

Exámenes intermedios (11-20)

2.1. Examen 11: Metodos de strings

Preguntas teoricas (5)

1. ¿Que metodo verifica si un string contiene solo digitos?
2. ¿Que metodo verifica si un string contiene solo letras?
3. ¿Que hace `.lower()`?
4. ¿Que hace `.strip()`?
5. ¿Como se separa un string en una lista de palabras?

Ejercicios practicos (5)

Ejercicio

1. Pide una cadena y muestra si es completamente alfabética.

Ejercicio

2. Pide un numero como string y verifica si es entero positivo.

Ejercicio

3. Pide una palabra y verifica si esta en mayúsculas.

Ejercicio

4. Pide una frase y cuenta cuantas palabras tiene (usa `split`).

Ejercicio

5. Pide una frase y reemplaza todos los espacios por guiones.

Soluciones

Solucion

Teóricas:

1. `.isdigit()`
2. `.isalpha()`
3. Convierte a minúsculas.
4. Elimina espacios al inicio y final.
5. `.split()`

Prácticas:

```
1 # 1
2 texto = input("Texto: ")
3 if texto.isalpha():
4     print("Solo letras")
5 else:
6     print("Contiene otros caracteres")
7
8 # 2
9 num = input("Numero: ")
10 if num.isdigit():
11     print("Entero positivo")
12 else:
13     print("No es entero positivo")
14
15 # 3
16 palabra = input("Palabra: ")
17 if palabra.isupper():
18     print("Esta en mayúsculas")
19
20 # 4
21 frase = input("Frase: ")
22 palabras = frase.split()
23 print("Cantidad de palabras:", len(palabras))
24
25 # 5
26 frase = input("Frase: ")
27 print(frase.replace(" ", "-"))
```

2.2. Examen 12: Matrices

Preguntas teóricas (5)

1. ¿Cómo se crea una matriz 3x3 en Python?

2. ¿Como se accede al elemento de la fila 1, columna 2?
3. ¿Como se recorre una matriz?
4. ¿Como se suma toda una matriz?
5. ¿Como se crea una matriz identidad 3x3?

Ejercicios practicos (5)

Ejercicio

1. Crea la matriz $\begin{bmatrix} 1,2,3 \\ 4,5,6 \\ 7,8,9 \end{bmatrix}$ y muéstrala.

Ejercicio

2. Suma todos los elementos de la matriz.

Ejercicio

3. Encuentra el valor maximo de la matriz.

Ejercicio

4. Crea una funcion que reciba una matriz y devuelva la suma de la diagonal principal.

Ejercicio

5. Crea una matriz identidad 4x4.

Soluciones

Solucion

Teóricas:

1. `matriz = [[0]*3 for i in range(3)]` o con valores fijos.
2. `matriz[1][2]`
3. `for fila in matriz: for elem in fila:`
4. Suma usando dos bucles anidados.
5. `[[1 if i==j else 0 for j in range(3)] for i in range(3)]`

Prácticas:

```
1 # 1
2 mat = [[1,2,3],[4,5,6],[7,8,9]]
3 for f in mat:
4     print(f)
5
6 # 2
7 suma = 0
8 for f in mat:
9     for e in f:
10        suma += e
11 print("Suma total:", suma)
12
13 # 3
14 maximo = mat[0][0]
15 for f in mat:
16     for e in f:
17         if e > maximo:
18             maximo = e
19 print("Maximo:", maximo)
20
21 # 4
22 def suma_diagonal(mat):
23     suma = 0
24     for i in range(len(mat)):
25         suma += mat[i][i]
26     return suma
27 print(suma_diagonal(mat))
28
29 # 5
30 identidad = [[1 if i==j else 0 for j in range(4)] for i in
31               range(4)]
32 for f in identidad:
33     print(f)
```

2.3. Examen 13: Comprension de listas

Preguntas teoricas (5)

1. ¿Que es la comprension de listas?
2. ¿Como se genera una lista con los cuadrados del 1 al 10?
3. ¿Como se filtran los numeros pares de una lista usando comprension?
4. ¿Que hace `[i for i in range(5) if i % 2 == 0]`?
5. ¿Se puede usar comprension con diccionarios?

Ejercicios practicos (5)

Ejercicio

1. Genera una lista con los cuadrados del 1 al 10.

Ejercicio

2. Genera una lista con los numeros pares del 1 al 20.

Ejercicio

3. Dada una lista de palabras, crea una nueva lista con sus longitudes.

Ejercicio

4. Dada una frase, genera una lista de las vocales que aparecen.

Ejercicio

5. Genera una lista de 10 numeros aleatorios usando comprension.

Soluciones

Solucion

Teoricas:

1. Forma compacta de crear listas a partir de una secuencia y opcionalmente filtrar.
2. `[i**2 for i in range(1,11)]`
3. `[i for i in lista if i% 2 == 0]`
4. Lista de numeros pares entre 0 y 4: `[0,2,4]`.
5. Si, con dict comprehension: `k:v for ....`

Practicas:

```
1 # 1
2 cuadrados = [i**2 for i in range(1,11)]
3 print(cuadrados)
4
5 # 2
6 pares = [i for i in range(1,21) if i % 2 == 0]
7 print(pares)
8
9 # 3
10 palabras = ["hola", "mundo", "python"]
11 longitudes = [len(p) for p in palabras]
12 print(longitudes)
13
14 # 4
15 frase = "hola mundo"
16 vocales = [letra for letra in frase if letra in "aeiou"]
17 print(vocales)
18
19 # 5
20 import random
21 aleatorios = [random.randint(1,100) for _ in range(10)]
22 print(aleatorios)
```

2.4. Examen 14: Carrito de compras

Preguntas teoricas (5)

1. ¿Que estructura de datos usamos para guardar los productos del carrito?
2. ¿Que informacion tiene cada producto?
3. ¿Como se calcula el subtotal de un producto?
4. ¿Como se aplica un descuento del 10 %?

5. ¿Que metodo se usa para agregar un producto a la lista del carrito?

Ejercicios practicos (5)

Ejercicio

1. Escribe la funcion `agregar_producto()` que pide nombre, precio y cantidad.

Ejercicio

2. Escribe la funcion `ver_carrito()` que muestra todos los productos.

Ejercicio

3. Escribe la funcion `calcular_total()` que suma y aplica descuento si ≥ 10000 .

Ejercicio

4. Escribe el menu principal.

Ejercicio

- 5.Codigo completo del carrito.

Soluciones

Solucion

Teóricas:

1. Lista de diccionarios.
2. nombre, precio, cantidad.
3. precio * cantidad.
4. total = total * 0.90
5. carrito.append(producto)

Prácticas:

```
1 carrito = []
2
3 def agregar():
4     nombre = input("Nombre: ")
5     precio = float(input("Precio: $"))
6     cantidad = int(input("Cantidad: "))
7     carrito.append({"nombre": nombre, "precio": precio, "
8         cantidad": cantidad})
9     print("Producto agregado.")
10
11 def ver():
12     if not carrito:
13         print("Carrito vacío.")
14         return
15     for i, p in enumerate(carrito, 1):
16         subtotal = p["precio"] * p["cantidad"]
17         print(f"{i}. {p['nombre']} - ${p['precio']} x {p['
18             cantidad']} = ${subtotal:.2f}")
19
20 def total():
21     if not carrito:
22         print("Carrito vacío.")
23         return
24     suma = 0
25     for p in carrito:
26         suma += p["precio"] * p["cantidad"]
27     print(f"Subtotal: ${suma:.2f}")
28     if suma > 10000:
29         desc = suma * 0.10
30         suma -= desc
31         print(f"Descuento 10%: -${desc:.2f}")
32     print(f"Total: ${suma:.2f}")
33
34 while True:
35     print("\n--- CARRITO ---")
36     print("1. Agregar producto")
37     print("2. Ver carrito")
38     print("3. Calcular total")
39     print("4. Salir")
40     op = input("Opcion: ")
41     if op == "1":
42         agregar()
```

2.5. Examen 15: Modulo os basico

Preguntas teoricas (5)

1. ¿Que hace `os.getcwd()`?
2. ¿Que hace `os.listdir()`?
3. ¿Como se crea una carpeta con `os`?
4. ¿Como se renombra un archivo?
5. ¿Que hace `os.path.exists()`?

Ejercicios practicos (5)

Ejercicio

1. Muestra el directorio actual.

Ejercicio

2. Muestra todos los archivos y carpetas del directorio actual.

Ejercicio

3. Crea una carpeta llamada “prueba”.

Ejercicio

4. Verifica si la carpeta “prueba” existe.

Ejercicio

5. Crea un archivo de texto con `with open` y escribe algo.

Soluciones

Solucion

Teoricas:

1. Obtiene la ruta del directorio actual de trabajo.
2. Lista el contenido del directorio.
3. `os.mkdir(nombre)` o `os.makedirs()`
4. `os.rename(viejo, nuevo)`
5. Verifica si una ruta existe (archivo o carpeta).

Practicas:

```
1 import os
2
3 # 1
4 print(os.getcwd())
5
6 # 2
7 print(os.listdir())
8
9 # 3
10 os.mkdir("prueba")
11
12 # 4
13 if os.path.exists("prueba"):
14     print("Existe")
15 else:
16     print("No existe")
17
18 # 5
19 with open("archivo.txt", "w") as f:
20     f.write("Hola mundo")
```

2.6. Examen 16: Ahorcado

Preguntas teoricas (5)

1. ¿Cuántos intentos tiene el jugador en el ahorcado?
2. ¿Cómo se muestra la palabra oculta (con guiones)?
3. ¿Cómo se verifica si el jugador ya ganó?
4. ¿Qué método de string se usa para validar que el input sea una letra?
5. ¿Cómo se manejan las letras repetidas?

Ejercicios practicos (5)

Ejercicio

1. Funcion `elegir_palabra()` que elige una palabra de una lista.

Ejercicio

2. Función `mostrar_progreso(palabra, acertadas)` que devuelve el string con guiones.

Ejercicio

3. Funcion `verificar_ganaste(palabra, acertadas)`.

Ejercicio

4. Bucle principal con 6 intentos.

Ejercicio

- 5.Codigo completo del juego.

Soluciones

Solucion

Teóricas:

1. Generalmente 6.
2. Reemplazando las letras no adivinadas por “_”.
3. Verificando que no quede ningún “_” en la palabra mostrada.
4. `.isalpha()`
5. Se guardan en una lista de letras acertadas y se evitan las repetidas.

Prácticas:

```
1 import random
2
3 def elegir_palabra():
4     palabras = ["python", "programacion", "computadora", "
5         ahorcado"]
6     return random.choice(palabras)
7
8 def mostrar_progreso(palabra, acertadas):
9     res = ""
10    for letra in palabra:
11        if letra in acertadas:
12            res += letra + " "
13        else:
14            res += "_ "
15    return res
16
17 def verificar_ganaste(palabra, acertadas):
18     for letra in palabra:
19         if letra not in acertadas:
20             return False
21     return True
22
23 def jugar():
24     palabra = elegir_palabra()
25     acertadas = []
26     fallidas = []
27     intentos = 6
28
29     while intentos > 0:
30         print("\nPalabra:", mostrar_progreso(palabra,
31             acertadas))
32         print("Letras fallidas:", fallidas)
33         print("Intentos restantes:", intentos)
34         letra = input("Letra: ").lower()
35
36         if len(letra) != 1 or not letra.isalpha():
37             print("Ingrese una sola letra.")
38             continue
39         if letra in acertadas or letra in fallidas:
40             print("Ya usaste esa letra.")
41             continue
```

2.7. Examen 17: Generador de contraseñas

Preguntas teoricas (5)

1. ¿Que modulo se usa para generar valores aleatorios?
2. ¿Que modulo proporciona conjuntos de caracteres como `ascii_letters`?
3. ¿Que funcion del modulo random elige un elemento al azar de una lista?
4. ¿Como se convierten una lista de caracteres en un string?
5. ¿Por que es recomendable incluir simbolos en las contraseñas?

Ejercicios practicos (5)

Ejercicio

1. Genera una contraseña de 8 letras (mayusculas/minusculas).

Ejercicio

2. Genera una contraseña de 6 digitos.

Ejercicio

3. Genera una contraseña de 10 caracteres (letras+digitos).

Ejercicio

4. Genera una contraseña de 12 caracteres (letras+digitos+simbolos).

Ejercicio

5. Genera una contraseña que asegure al menos una mayuscula, un digito y un simbolo.

Soluciones

Solucion

Teóricas:

1. random
2. string
3. random.choice()
4. ''.join(lista)
5. Aumentan la complejidad y la seguridad.

Prácticas:

```
1 import random
2 import string
3
4 # 1
5 long = 8
6 car = string.ascii_letters
7 pwd = ''.join(random.choice(car) for _ in range(long))
8 print(pwd)
9
10 # 2
11 car = string.digits
12 pwd = ''.join(random.choice(car) for _ in range(6))
13 print(pwd)
14
15 # 3
16 car = string.ascii_letters + string.digits
17 pwd = ''.join(random.choice(car) for _ in range(10))
18 print(pwd)
19
20 # 4
21 car = string.ascii_letters + string.digits + string.
    punctuation
22 pwd = ''.join(random.choice(car) for _ in range(12))
23 print(pwd)
24
25 # 5
26 def generar_segura(long=10):
27     mayus = random.choice(string.ascii_uppercase)
28     digito = random.choice(string.digits)
29     simbolo = random.choice(string.punctuation)
30     resto = ''.join(random.choice(string.ascii_letters +
    string.digits + string.punctuation)
    for _ in range(long-3))
31     lista = list(mayus + digito + simbolo + resto)
32     random.shuffle(lista)
33     return ''.join(lista)
34
35 print(generar_segura())
```

2.8. Examen 18: Sistema de notas

Preguntas teoricas (5)

1. ¿Que estructura de datos se usa para guardar las notas de cada alumno?
2. ¿Como se asocian las notas con el nombre del alumno?
3. ¿Como se calcula el promedio de un alumno?
4. ¿Que metodo se usa para agregar una nota a la lista de notas de un alumno?
5. ¿Por que es necesario convertir el promedio a string para mostrarlo?

Ejercicios practicos (5)

Ejercicio

1. Funcion `agregar_alumno()` que pide nombre y crea una lista vacia de notas.

Ejercicio

2. Funcion `agregar_nota()` que pide nombre y nota y la agrega.

Ejercicio

3. Funcion `mostrar_promedios()` que muestra el promedio de cada alumno.

Ejercicio

4. Funcion `mostrar_alumnos()` que lista los nombres.

Ejercicio

5. Menu principal.

Soluciones

Solucion

Teóricas:

1. Diccionario donde la clave es el nombre y el valor es una lista de notas.
2. Con un diccionario: `notas[nombre] = []`
3. Suma de notas / cantidad de notas.
4. `notas[nombre].append(nota)`
5. Porque el operador `+` no puede concatenar string con numero.

Prácticas:

```
1 alumnos = []
2 notas = {}
3
4 def agregar_alumno():
5     nom = input("Nombre: ")
6     alumnos.append(nom)
7     notas[nom] = []
8     print("Alumno agregado.")
9
10 def agregar_nota():
11     nom = input("Alumno: ")
12     if nom in notas:
13         nota = float(input("Nota: "))
14         notas[nom].append(nota)
15         print("Nota agregada.")
16     else:
17         print("Alumno no existe.")
18
19 def mostrar_promedios():
20     for nom, lista in notas.items():
21         if lista:
22             prom = sum(lista) / len(lista)
23         else:
24             prom = 0
25         print(f"{nom}: {prom:.2f}")
26
27 def mostrar_alumnos():
28     for i, nom in enumerate(alumnos):
29         print(f"{i}. {nom}")
30
31 while True:
32     print("\n1. Agregar alumno")
33     print("2. Agregar nota")
34     print("3. Ver promedios")
35     print("4. Ver alumnos")
36     print("5. Salir")
37     op = input("Opcion: ")
38     if op == "1":
39         agregar_alumno()
40     elif op == "2":
41         agregar_nota()
```

2.9. Examen 19: Try/Except

Preguntas teoricas (5)

1. ¿Que hace el bloque `try/except`?
2. ¿Que tipo de error captura `except ValueError`?
3. ¿Que pasa si no se usa `try/except` y ocurre un error?
4. ¿Como se captura cualquier tipo de error?
5. ¿Para que sirve el bloque `else` en un `try/except`?

Ejercicios practicos (5)

Ejercicio

1. Pide un numero entero; si no es numero, muestra error.

Ejercicio

2. Pide dos numeros y divide; captura division por cero.

Ejercicio

3. Accede a una lista con indice pedido; captura `IndexError`.

Ejercicio

4. Convierte un string a int; captura `ValueError`.

Ejercicio

5. Abre un archivo que no existe; captura `FileNotFoundError`.

Soluciones

Solucion

Teóricas:

1. Captura errores para que no terminen el programa abruptamente.
2. Errores de conversión de tipo o valor inapropiado.
3. El programa se detiene y muestra un traceback.
4. `except:` (sin especificar tipo)
5. Se ejecuta si no ocurrió ninguna excepción.

Prácticas:

```
1  # 1
2  try:
3      n = int(input("Numero: "))
4      print("OK")
5  except ValueError:
6      print("No es un numero")
7
8  # 2
9  try:
10     a = float(input("A: "))
11     b = float(input("B: "))
12     print(a / b)
13 except ZeroDivisionError:
14     print("No se puede dividir por cero")
15
16 # 3
17 lista = [10,20,30]
18 try:
19     idx = int(input("Indice: "))
20     print(lista[idx])
21 except IndexError:
22     print("Indice fuera de rango")
23 except ValueError:
24     print("Indice debe ser numero")
25
26 # 4
27 try:
28     texto = "123a"
29     numero = int(texto)
30 except ValueError:
31     print("No se puede convertir")
32
33 # 5
34 try:
35     with open("no_existe.txt", "r") as f:
36         print(f.read())
37 except FileNotFoundError:
38     print("El archivo no existe")
```

2.10. Examen 20: Agenda de contactos

Preguntas teoricas (5)

1. ¿Que estructura de datos es ideal para una agenda de contactos?
2. ¿Como se agrega un contacto?
3. ¿Como se busca el telefono de un contacto dado su nombre?
4. ¿Como se elimina un contacto?
5. ¿Como se listan todos los contactos?

Ejercicios practicos (5)

Ejercicio

1. Funcion `agregar()` que pide nombre y telefono.

Ejercicio

2. Funcion `buscar()` que muestra el telefono.

Ejercicio

3. Funcion `eliminar()` que borra un contacto.

Ejercicio

4. Funcion `listar()` que muestra todos.

Ejercicio

5. Menu principal.

Soluciones

Solucion

Teóricas:

1. Diccionario (clave=nombre, valor=telefono).
2. agenda[nombre] = telefono
3. agenda.get(nombre) o agenda[nombre].
4. del agenda[nombre]
5. for nombre, tel in agenda.items(): print(nombre, tel)

Prácticas:

```
1 agenda = {}
2
3 def agregar():
4     nom = input("Nombre: ")
5     tel = input("Telefono: ")
6     agenda[nom] = tel
7     print("Contacto agregado.")
8
9 def buscar():
10    nom = input("Nombre a buscar: ")
11    if nom in agenda:
12        print(f"{nom}: {agenda[nom]}")
13    else:
14        print("No existe.")
15
16 def eliminar():
17    nom = input("Nombre a eliminar: ")
18    if nom in agenda:
19        del agenda[nom]
20        print("Eliminado.")
21    else:
22        print("No existe.")
23
24 def listar():
25    if not agenda:
26        print("Agenda vacia.")
27    else:
28        for nom, tel in agenda.items():
29            print(f"{nom}: {tel}")
30
31 while True:
32     print("\n1. Agregar")
33     print("2. Buscar")
34     print("3. Eliminar")
35     print("4. Listar")
36     print("5. Salir")
37     op = input("Opcion: ")
38     if op == "1":
39         agregar()
40     elif op == "2":
41         buscar()
```

Capítulo 3

Exámenes avanzados (21-30)

3.1. Examen 21: Bucles anidados

Preguntas teoricas (5)

1. ¿Que es un bucle anidado?
2. ¿Para que sirven los bucles anidados?
3. ¿Como se muestra una matriz con dos for?
4. ¿Que hace el primer for y que el segundo?
5. ¿Como se genera una tabla de multiplicar completa (1 al 10) con dos for?

Ejercicios practicos (5)

Ejercicio

1. Muestra un triangulo de asteriscos de altura 5 (con for anidado o con * multiplicado).

Ejercicio

2. Muestra la tabla de multiplicar del 1 al 5 usando dos for.

Ejercicio

3. Muestra una matriz 3x3 con numeros del 1 al 9.

Ejercicio

4. Muestra un cuadrado de asteriscos 5x5.

Ejercicio

5. Muestra una piramide de numeros (1, 12, 123, 1234, 12345).

Soluciones

Solucion

Teóricas:

1. Un bucle dentro de otro bucle.
2. Para trabajar con estructuras bidimensionales (matrices, tablas).
3. Primer for para filas, segundo for para columnas.
4. El primero recorre filas, el segundo recorre elementos de esa fila.
5. `for i in range(1,11): for j in range(1,11): print(i*j)`

Prácticas:

```
1 # 1
2 for i in range(1,6):
3     print("*" * i)
4
5 # 2
6 for i in range(1,6):
7     print(f"Tabla del {i}:")
8     for j in range(1,11):
9         print(f"{i} x {j} = {i*j}")
10    print()
11
12 # 3
13 for i in range(3):
14     for j in range(3):
15         print(i*3 + j + 1, end=" ")
16     print()
17
18 # 4
19 for i in range(5):
20     print("*" * 5)
21
22 # 5
23 for i in range(1,6):
24     for j in range(1, i+1):
25         print(j, end=" ")
26     print()
```

3.2. Examen 22: Piedra Papel Tijera

Preguntas teóricas (5)

1. ¿Cuántas opciones tiene el juego básico?
2. ¿Cómo se elige la opción de la computadora?

3. ¿Cual es la regla de Piedra?
4. ¿Como se determina el ganador?
5. ¿Como se implementa un mejor de 3 rondas?

Ejercicios practicos (5)

Ejercicio

1. Funcion `obtener_cpu()` que devuelve una opcion al azar.

Ejercicio

2. Funcion `ganador(jugador, cpu)` que devuelve “jugador”, “cpu” o “empate”.

Ejercicio

3. Bucle para jugar una partida.

Ejercicio

4. Mejor de 3 rondas con contador de puntos.

Ejercicio

- 5.Codigo completo.

Soluciones

Solucion

Teóricas:

1. 3: piedra, papel, tijera.
2. `random.choice(["piedra","papel","tijera"])`
3. Piedra aplasta tijera.
4. Comparando las opciones con las reglas.
5. Se juegan hasta que alguien llegue a 3 puntos.

Prácticas:

```
1 import random
2
3 def obtener_cpu():
4     return random.choice(["piedra","papel","tijera"])
5
6 def ganador(jugador, cpu):
7     if jugador == cpu:
8         return "empate"
9     if (jugador == "piedra" and cpu == "tijera") or \
10        (jugador == "papel" and cpu == "piedra") or \
11        (jugador == "tijera" and cpu == "papel"):
12         return "jugador"
13     return "cpu"
14
15 puntos_j = 0
16 puntos_c = 0
17
18 while puntos_j < 3 and puntos_c < 3:
19     jugador = input("Elige piedra, papel o tijera: ").lower()
20     while jugador not in ["piedra","papel","tijera"]:
21         jugador = input("Invalido. Elige: ")
22     cpu = obtener_cpu()
23     print(f"CPU eligio: {cpu}")
24     res = ganador(jugador, cpu)
25     if res == "jugador":
26         puntos_j += 1
27         print("Ganaste la ronda!")
28     elif res == "cpu":
29         puntos_c += 1
30         print("Perdiste la ronda!")
31     else:
32         print("Empate!")
33     print(f"Marcador: {puntos_j} - {puntos_c}")
34
35 if puntos_j == 3:
36     print("Ganaste el juego!")
37 else:
38     print("Perdiste el juego!")
```

3.3. Examen 23: Biblioteca

Preguntas teoricas (5)

1. ¿Que informacion tiene un libro en una biblioteca?
2. ¿Como se presta un libro?
3. ¿Como se devuelve?
4. ¿Que estructura se usa para guardar los libros?
5. ¿Como se busca un libro por titulo?

Ejercicios practicos (5)

Ejercicio

1. Diccionario de libros (titulo: autor, estado).

Ejercicio

2. Funcion `agregar()` que pide titulo y autor.

Ejercicio

3. Funcion `prestar()` que cambia estado a “prestado”.

Ejercicio

4. Funcion `devolver()` que cambia a “disponible”.

Ejercicio

5. Funcion `mostrar()` que lista todos.

Soluciones

Solucion

Teóricas:

1. Título, autor, estado (disponible/prestado).
2. Cambiar estado a “prestado”.
3. Cambiar estado a “disponible”.
4. Diccionario o lista de diccionarios.
5. Recorrer el diccionario o usar `in`.

Prácticas:

```
1 biblioteca = {}
2
3 def agregar():
4     titulo = input("Título: ")
5     autor = input("Autor: ")
6     biblioteca[titulo] = {"autor": autor, "estado": "disponible"}
7     print("Libro agregado.")
8
9 def prestar():
10    titulo = input("Título a prestar: ")
11    if titulo in biblioteca and biblioteca[titulo]["estado"] == "disponible":
12        biblioteca[titulo]["estado"] = "prestado"
13        print("Libro prestado.")
14    else:
15        print("No disponible o no existe.")
16
17 def devolver():
18    titulo = input("Título a devolver: ")
19    if titulo in biblioteca:
20        biblioteca[titulo]["estado"] = "disponible"
21        print("Devuelto.")
22    else:
23        print("No existe.")
24
25 def mostrar():
26    if not biblioteca:
27        print("Biblioteca vacía.")
28    else:
29        for tit, datos in biblioteca.items():
30            print(f"{tit} - {datos['autor']} ({datos['estado']})")
31
32 while True:
33     print("\n1. Agregar libro")
34     print("2. Prestar libro")
35     print("3. Devolver libro")
36     print("4. Mostrar libros")
37     print("5. Salir")
38     op = input("Opción: ")
39     if op == "1":
40         agregar()
41     elif op == "2":
42         prestar()
43     elif op == "3":
44         devolver()
45     elif op == "4":
46         mostrar()
47     elif op == "5":
48         break
```

3.4. Examen 24: Recursividad

Preguntas teoricas (5)

1. ¿Que es una funcion recursiva?
2. ¿Que dos partes debe tener una funcion recursiva?
3. ¿Que es el caso base?
4. ¿Que desventaja tiene la recursion frente a la iteracion?
5. ¿Como se calcula el factorial de forma recursiva?

Ejercicios practicos (5)

Ejercicio

1. Funcion recursiva que calcule el factorial.

Ejercicio

2. Funcion recursiva que calcule la suma de 1 a n.

Ejercicio

3. Funcion recursiva que cuente letras de una palabra.

Ejercicio

4. Funcion recursiva que invierta una cadena.

Ejercicio

5. Funcion recursiva para la serie de Fibonacci.

Soluciones

Solucion

Teóricas:

1. Funcion que se llama a si misma.
2. Caso base y caso recursivo.
3. Condicion que detiene la recursion.
4. Mayor consumo de memoria (pila de llamadas).
5. $\text{factorial}(n) = n * \text{factorial}(n-1)$ con caso base $\text{factorial}(0)=1$.

Prácticas:

```
1 # 1
2 def factorial(n):
3     if n <= 1:
4         return 1
5     return n * factorial(n-1)
6
7 # 2
8 def suma_hasta(n):
9     if n == 1:
10        return 1
11    return n + suma_hasta(n-1)
12
13 # 3
14 def contar_letras(palabra):
15     if palabra == "":
16         return 0
17     return 1 + contar_letras(palabra[1:])
18
19 # 4
20 def invertir(cad):
21     if cad == "":
22         return ""
23     return invertir(cad[1:]) + cad[0]
24
25 # 5
26 def fibonacci(n):
27     if n <= 1:
28         return n
29     return fibonacci(n-1) + fibonacci(n-2)
30
31 print(factorial(5))
32 print(suma_hasta(5))
33 print(contar_letras("hola"))
34 print(invertir("python"))
35 print([fibonacci(i) for i in range(10)])
```

3.5. Examen 25: Lista de tareas

Preguntas teoricas (5)

1. ¿Que estructura usar para guardar tareas?
2. ¿Que informacion tiene cada tarea?
3. ¿Como se marca una tarea como completada?
4. ¿Como se elimina una tarea?
5. ¿Como se muestran las tareas con su estado?

Ejercicios practicos (5)

Ejercicio

1. Funcion `agregar()` que pide descripcion y la agrega como no completada.

Ejercicio

2. Funcion `mostrar()` que lista con indice y estado.

Ejercicio

3. Funcion `completar()` que cambia estado a True.

Ejercicio

4. Funcion `eliminar()` que borra por indice.

Ejercicio

5. Menu principal.

Soluciones

Solucion

Teóricas:

1. Lista de diccionarios.
2. descripcion, completada (bool).
3. Cambiar el campo completada a True.
4. pop(indice)
5. Mostrar [X] o [] segun el estado.

Prácticas:

```
1 tareas = []
2
3 def agregar():
4     desc = input("Tarea: ")
5     tareas.append({"descripcion": desc, "completada": False})
6     print("Tarea agregada.")
7
8 def mostrar():
9     if not tareas:
10        print("No hay tareas.")
11    else:
12        for i, t in enumerate(tareas):
13            estado = "X" if t["completada"] else " "
14            print(f"{i}. [{estado}] {t['descripcion']}")
15
16 def completar():
17     mostrar()
18     try:
19         idx = int(input("Numero de tarea a completar: "))
20         if 0 <= idx < len(tareas):
21             tareas[idx]["completada"] = True
22             print("Completada!")
23         else:
24             print("Indice invalido.")
25     except ValueError:
26         print("Ingrese un numero.")
27
28 def eliminar():
29     mostrar()
30     try:
31         idx = int(input("Numero a eliminar: "))
32         if 0 <= idx < len(tareas):
33             eliminada = tareas.pop(idx)
34             print(f"Eliminada: {eliminada['descripcion']}")
35         else:
36             print("Indice invalido.")
37     except ValueError:
38         print("Ingrese un numero.")
39
40 while True:
41     print("\n1. Agregar tarea")
42     print("2. Mostrar tareas")
```

3.6. Examen 26: Conversor de unidades

Preguntas teoricas (5)

1. ¿Cuántos centímetros tiene un metro?
2. ¿Cuántos metros tiene un kilómetro?
3. ¿Cuántos minutos tiene una hora?
4. ¿Formula para convertir Celsius a Fahrenheit?
5. ¿Como se convierten dolares a pesos?

Ejercicios practicos (5)

Ejercicio

1. Funcion `m_a_cm(metros)`.

Ejercicio

2. Funcion `km_a_m(km)`.

Ejercicio

3. Funcion `h_a_min(horas)`.

Ejercicio

4. Funcion `c_a_f(celsius)`.

Ejercicio

5. Menu con opciones.

Soluciones

Solucion

Teoricas:

1. 100
2. 1000
3. 60
4. $F = C * 9/5 + 32$
5. dolar * tasa

Practicas:

```
1 def m_a_cm(m): return m * 100
2 def km_a_m(km): return km * 1000
3 def h_a_min(h): return h * 60
4 def c_a_f(c): return (c * 9/5) + 32
5 def usd_a_ars(usd, tasa=1200): return usd * tasa
6
7 while True:
8     print("\n1. Metros a cm")
9     print("2. Km a metros")
10    print("3. Horas a minutos")
11    print("4. Celsius a F")
12    print("5. USD a ARS")
13    print("6. Salir")
14    op = input("Opcion: ")
15    if op == "1":
16        v = float(input("Metros: "))
17        print(f"{v}m = {m_a_cm(v)}cm")
18    elif op == "2":
19        v = float(input("Km: "))
20        print(f"{v}km = {km_a_m(v)}m")
21    elif op == "3":
22        v = float(input("Horas: "))
23        print(f"{v}h = {h_a_min(v)}min")
24    elif op == "4":
25        v = float(input("Celsius: "))
26        print(f"{v}C = {c_a_f(v):.1f}F")
27    elif op == "5":
28        v = float(input("USD: "))
29        print(f"${v} USD = ${usd_a_ars(v):.2f} ARS")
30    elif op == "6":
31        break
```

3.7. Examen 27: Deteccion de errores

Preguntas teoricas (5)

1. ¿Que es un SyntaxError?
2. ¿Que es un NameError?
3. ¿Que es un TypeError?
4. ¿Que es un IndexError?
5. ¿Como se puede depurar un programa?

Ejercicios practicos (5)

Ejercicio

1. Corrige el siguiente codigo (falta convertir input):

```
1 edad = input("Edad: ")
2 if edad >= 18:
3     print("Mayor")
```

Ejercicio

2. Corrige (bucle infinito):

```
1 i = 0
2 while i < 5:
3     print(i)
```

Ejercicio

3. Corrige (error de indice):

```
1 lista = [1,2,3]
2 print(lista[3])
```

Ejercicio

4. Corrige (error de tipo):

```
1 num = "10"
2 suma = num + 5
```


Ejercicio

5. Corrige (error de sintaxis):

```
1 for i in range(5)
2     print(i)
```

Soluciones

Solucion

Teoricas:

1. Error de sintaxis (falta :, parentesis, etc.)
2. Variable no definida.
3. Operacion entre tipos incompatibles.
4. Indice fuera de rango.
5. Usar print() para ver valores, leer mensajes de error, etc.

Practicas:

```
1 # 1
2 edad = int(input("Edad: "))
3 if edad >= 18:
4     print("Mayor")
5
6 # 2
7 i = 0
8 while i < 5:
9     print(i)
10    i += 1
11
12 # 3
13 lista = [1,2,3]
14 print(lista[2]) # o lista[-1]
15
16 # 4
17 num = "10"
18 suma = int(num) + 5
19
20 # 5
21 for i in range(5):
22     print(i)
```

3.8. Examen 28: Variables globales y locales

Preguntas teoricas (5)

1. ¿Que es una variable global?
2. ¿Que es una variable local?
3. ¿Como se modifica una variable global dentro de una funcion?
4. ¿Que pasa si una variable local tiene el mismo nombre que una global?
5. ¿Que palabra clave se usa para indicar que usamos la variable global?

Ejercicios practicos (5)

Ejercicio

1. Crea una variable global `contador = 0` y una funcion que la incremente.

Ejercicio

2. Sin usar `global`, intenta modificarla y observa el error.

Ejercicio

3. Usa `global` para modificarla correctamente.

Ejercicio

4. Crea una variable local con el mismo nombre y muestra la diferencia.

Ejercicio

5. Escribe una funcion que use `global` y otra que no.

Soluciones

Solucion

Teóricas:

1. Definida fuera de cualquier función, visible en todo el programa.
2. Definida dentro de una función, solo visible dentro de ella.
3. Usando la palabra clave `global`.
4. Dentro de la función predomina la local.
5. `global`

Prácticas:

```
1 contador = 0
2
3 def incrementar():
4     global contador
5     contador += 1
6
7 def intentar_sin_global():
8     contador = 10 # crea una local, no modifica la global
9
10 def mostrar():
11     print(contador)
12
13 incrementar()
14 incrementar()
15 mostrar() # 2
16 intentar_sin_global()
17 mostrar() # sigue 2
18
19 def local_y_global():
20     x = 5 # local
21     global y
22     y = 10
23     print("Local x:", x)
24
25 y = 0
26 local_y_global()
27 print("Global y:", y)
```

3.9. Examen 29: Módulos

Preguntas teóricas (5)

1. ¿Cómo se importa un módulo completo?

2. ¿Como se importa solo una funcion de un modulo?
3. ¿Que hace `import math as m`?
4. ¿Que es un modulo nativo?
5. ¿Donde se guardan los modulos que uno crea?

Ejercicios practicos (5)

Ejercicio

1. Importa el modulo `math` y usa `sqrt()` para raiz cuadrada.

Ejercicio

2. Importa solo `randint` de `random`.

Ejercicio

3. Importa `datetime` y muestra la fecha actual.

Ejercicio

4. Crea un modulo propio (un archivo `.py`) con una funcion y usalo.

Ejercicio

5. Importa `math` con alias y usa `pi`.

Soluciones

Solucion

Teoricas:

1. `import modulo`
2. `from modulo import funcion`
3. Importa `math` con el alias `m`.
4. Viene incluido en Python (`os`, `random`, `math`, etc.)
5. En la misma carpeta que el script principal.

Practicas:

```
1  # 1
2  import math
3  print(math.sqrt(16))
4
5  # 2
6  from random import randint
7  print(randint(1,10))
8
9  # 3
10 import datetime
11 print(datetime.datetime.now())
12
13 # 4 (crear archivo mipropio.py con: def hola(): return "Hola
    ")
14 import mipropio
15 print(mipropio.hola())
16
17 # 5
18 import math as m
19 print(m.pi)
```

3.10. Examen 30: Supermercado con descuentos

Preguntas teoricas (5)

1. ¿Que es un descuento fijo y uno porcentual?
2. ¿Como se aplica un descuento del 20 %?
3. ¿Que estructura usamos para guardar productos con precio?
4. ¿Como se calcula el total de una compra?
5. ¿Que es una oferta 2x1?

Ejercicios practicos (5)**Ejercicio**

1. Crea un diccionario de productos (nombre: precio).

Ejercicio

2. Funcion para mostrar el menu.

Ejercicio

3. Funcion para agregar productos al carrito por nombre.

Ejercicio

4. Aplica descuento del 10 % si total >10000.

Ejercicio

5. Muestra el ticket final.

Soluciones

Solucion

Teóricas:

1. Fijo: resta un monto fijo; porcentual: resta un porcentaje del total.
2. $\text{total} = \text{total} * 0.80$
3. Diccionario o lista de diccionarios.
4. Suma de ($\text{precio} * \text{cantidad}$).
5. Llevarse dos productos por el precio de uno.

Prácticas:

```
1 productos = {
2     "pan": 2000,
3     "leche": 3000,
4     "carne": 15000,
5     "coca": 3500
6 }
7 carrito = []
8
9 def mostrar_menu():
10     for p, precio in productos.items():
11         print(f"{p}: ${precio}")
12
13 def agregar():
14     nombre = input("Producto: ")
15     if nombre in productos:
16         cantidad = int(input("Cantidad: "))
17         carrito.append({"nombre": nombre, "cantidad":
18             cantidad})
19         print("Agregado.")
20     else:
21         print("No existe.")
22
23 def ver_total():
24     total = 0
25     for item in carrito:
26         precio = productos[item["nombre"]]
27         subtotal = precio * item["cantidad"]
28         total += subtotal
29         print(f"{item['nombre']} x{item['cantidad']} = ${
30             subtotal}")
31     print(f"Subtotal: ${total}")
32     if total > 10000:
33         desc = total * 0.10
34         total -= desc
35         print(f"Descuento 10%: -${desc:.2f}")
36     print(f"Total: ${total:.2f}")
37
38 while True:
39     print("\n1. Ver productos")
40     print("2. Agregar al carrito")
41     print("3. Ver total")
42     print("4. Salir")
```